

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1102

COURSE OUTCOME COVERED

CO4:

implement object-oriented system layers using design principles and patterns, and integrate access and view layers with OODBMS (*BL5*)

Assessment Tool Weightage: 40%

QUESTIONS: 5

Total Marks:40

Annexure A

Industry Problem Solving Task

Code of Conduct:

I Tejaswini.M(192411079) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Tejaswini.M

Annexure A

Industry Problem Solving Task

1. Smart Banking and Financial Management System

A financial institution develops a **Smart Banking and Financial Management System** to manage customer accounts, fund transfers, loan processing, bill payments, and transaction history. The application follows a layered object-oriented architecture consisting of a presentation layer, service layer, domain layer, and data access layer connected to an OODBMS.

The system uses the **Factory Pattern** to create different account types, the **Strategy Pattern** for loan-interest and payment calculation policies, and the **Observer Pattern** for transaction and fraud alerts. A **Repository Pattern** is used to manage communication with persistent banking objects. The architecture is designed to support future services such as investment management and digital wallets.

Question:

Analyze how object-oriented principles, layered architecture, and design patterns contribute to the scalability, security, flexibility, and maintainability of the banking system. Explain how the application layers communicate with the OODBMS to store and retrieve persistent banking objects.

Introduction

The Smart Banking and Financial Management System manages **customer accounts, fund transfers, loan processing, bill payments, and transaction history**. The system uses a layered object-oriented architecture with a Presentation Layer, Service Layer, Domain Layer, Data Access Layer, and OODBMS.

The system uses four design patterns:

- **Factory Pattern:** Creates different account types.
- **Strategy Pattern:** Handles different loan-interest/payment policies.
- **Observer Pattern:** Provides transaction and fraud alerts.
- **Repository Pattern:** Manages persistent banking objects.

Object-Oriented Principles

Encapsulation

Banking data such as account number and balance is kept private and accessed through methods.

Abstraction

Interfaces such as AccountRepository, TransactionObserver, and LoanInterestStrategy hide implementation details.

Inheritance

SavingsAccount and CurrentAccount inherit common properties and operations from Account.

Polymorphism

Different implementations of LoanInterestStrategy can be used through the same interface.

SOLID Principles

The layered architecture supports **Single Responsibility**, **Open/Closed**, **Dependency Inversion**, and **low coupling**, making the system easier to maintain and extend.

Diagram 1: Layered Architecture

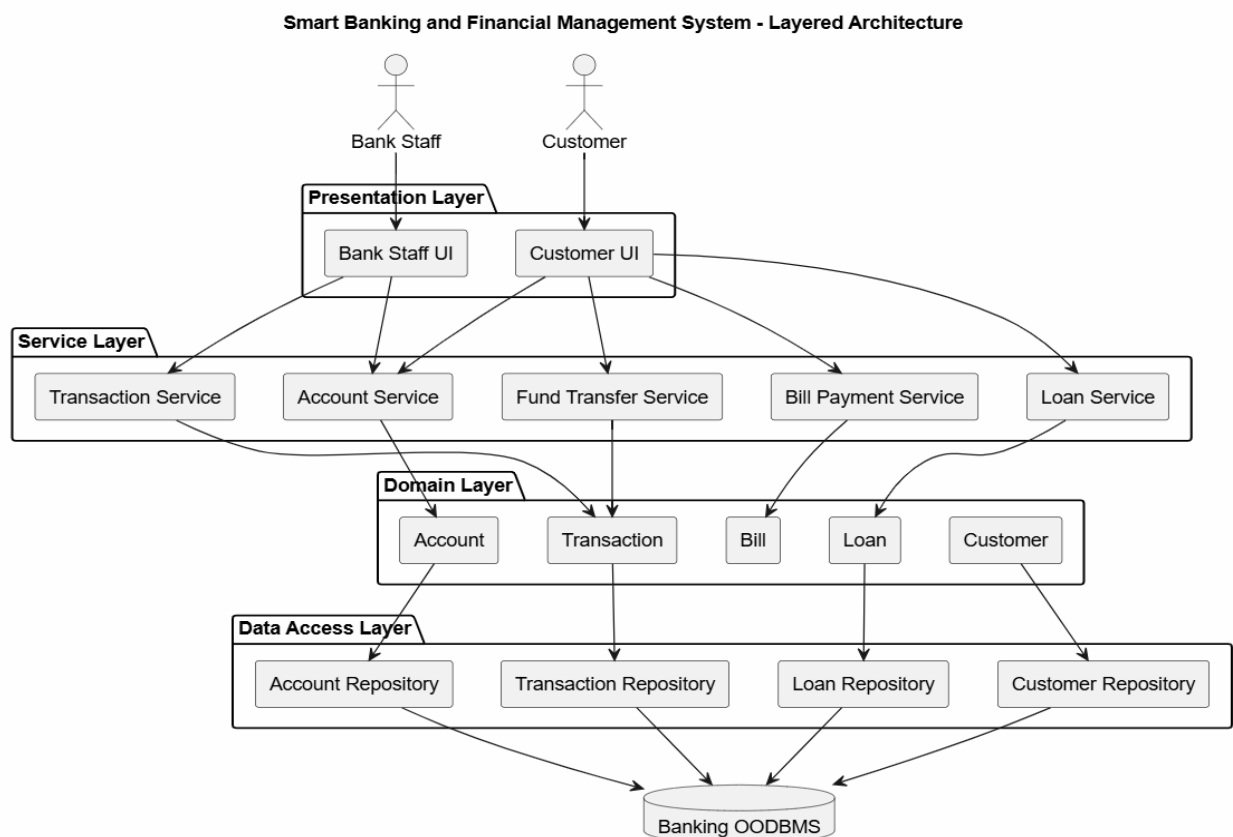


Diagram 2: Class Diagram with Design Patterns

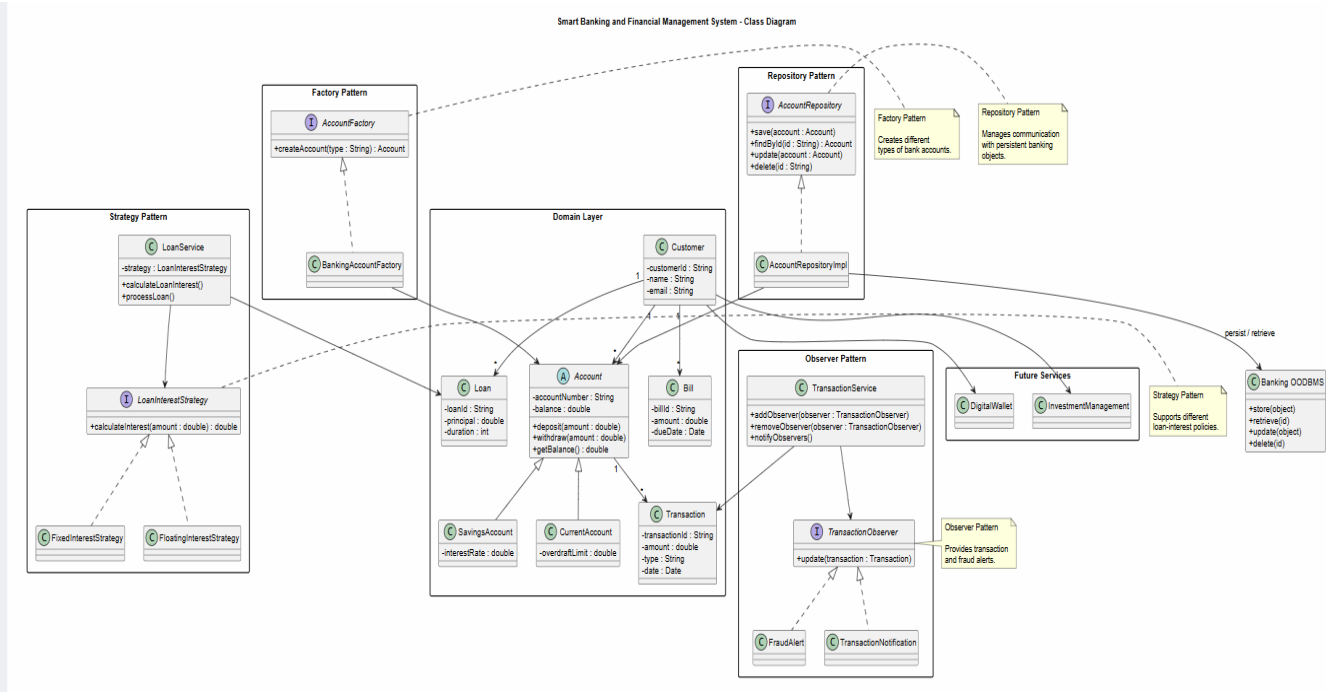


Diagram 3: Sequence Diagram

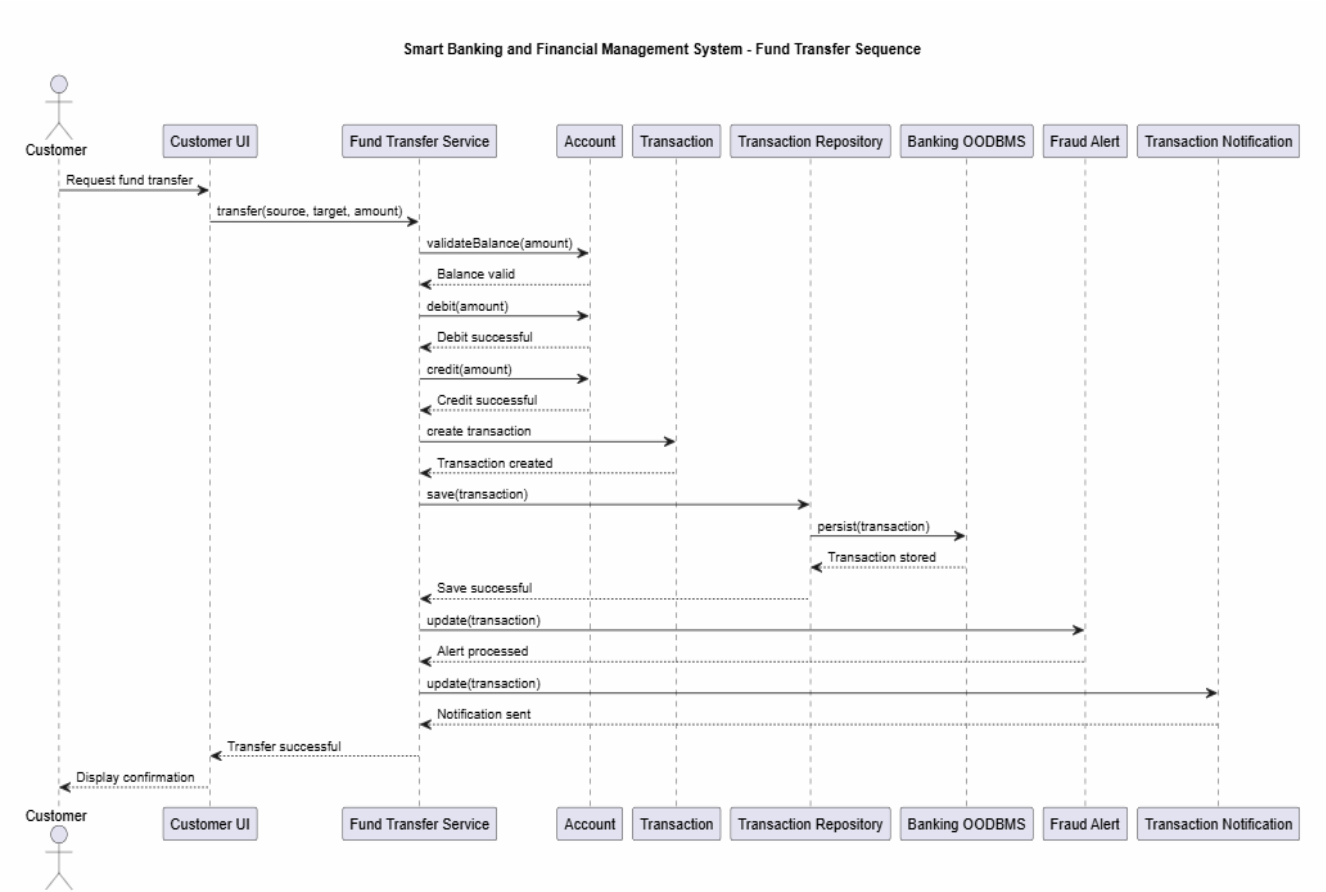


Diagram 4: OODBMS Persistence and Retrieval

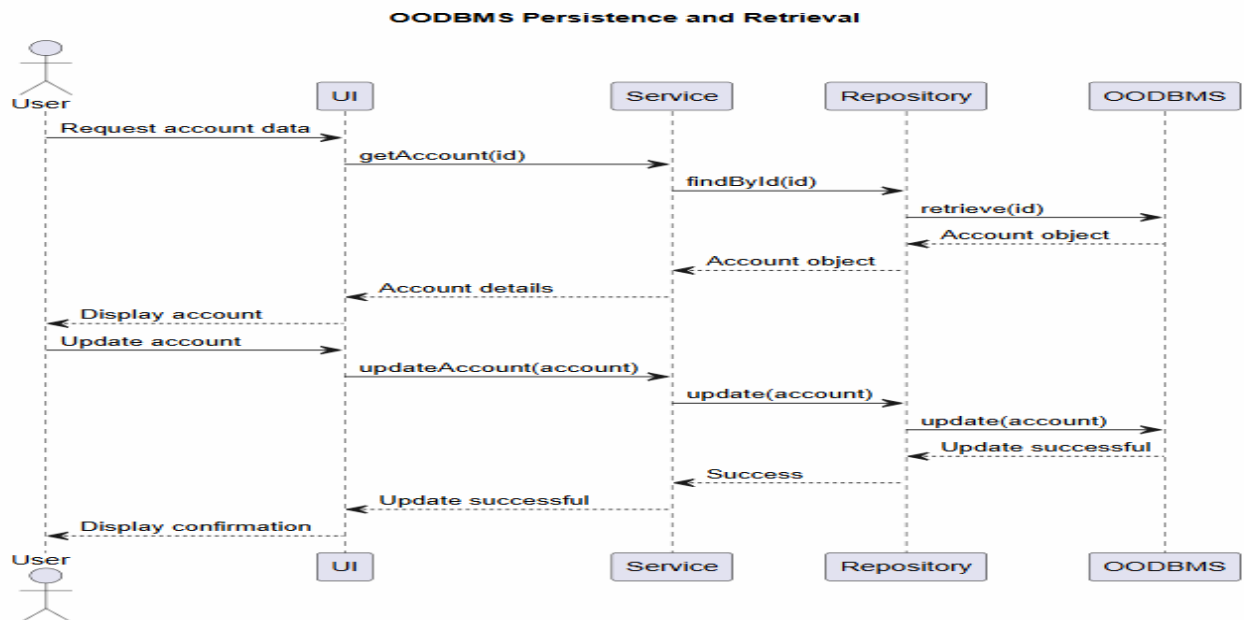


Diagram 5: Use Case Diagram

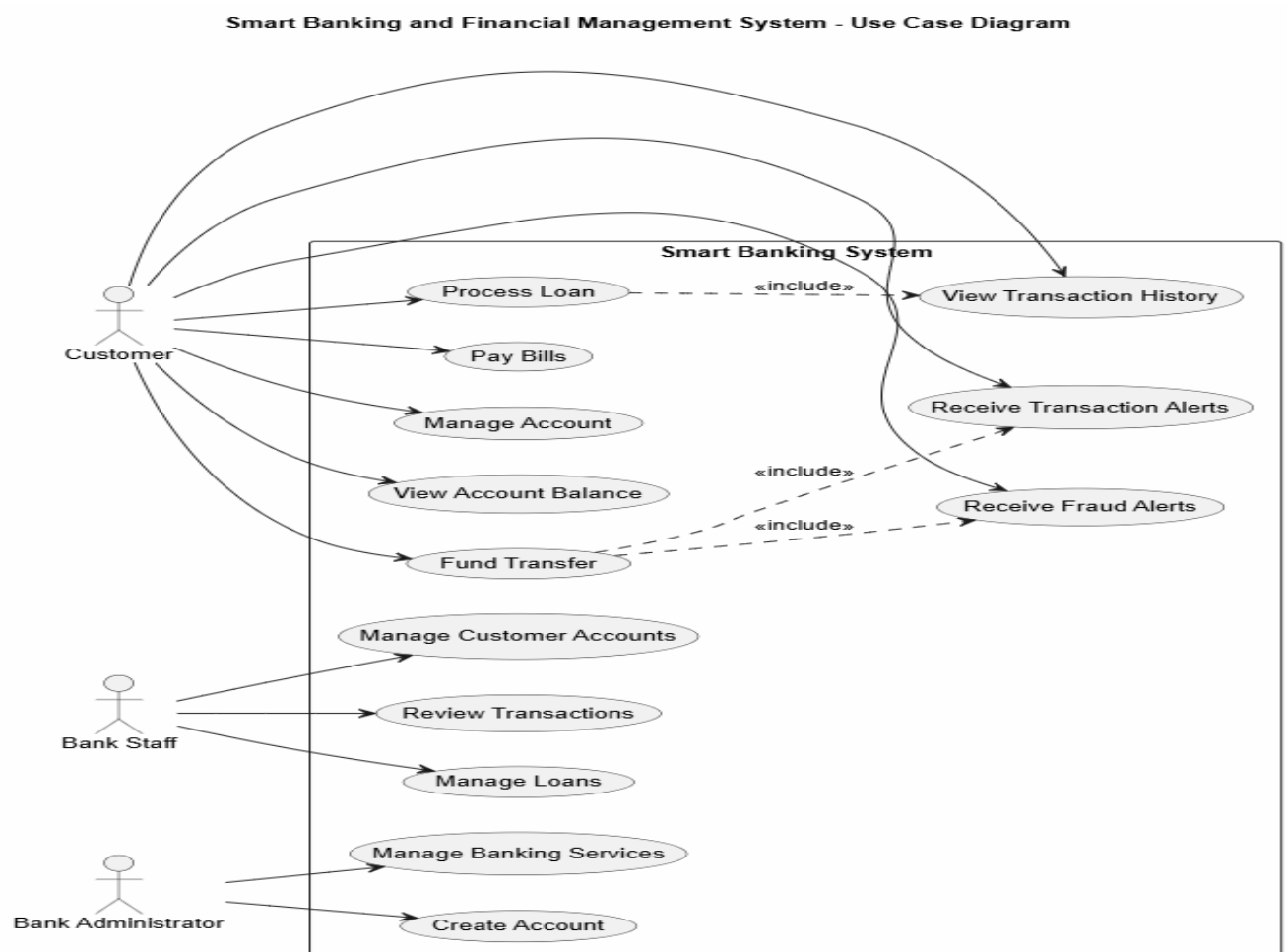


Diagram 6: Activity Diagram, Fund Transfer Process

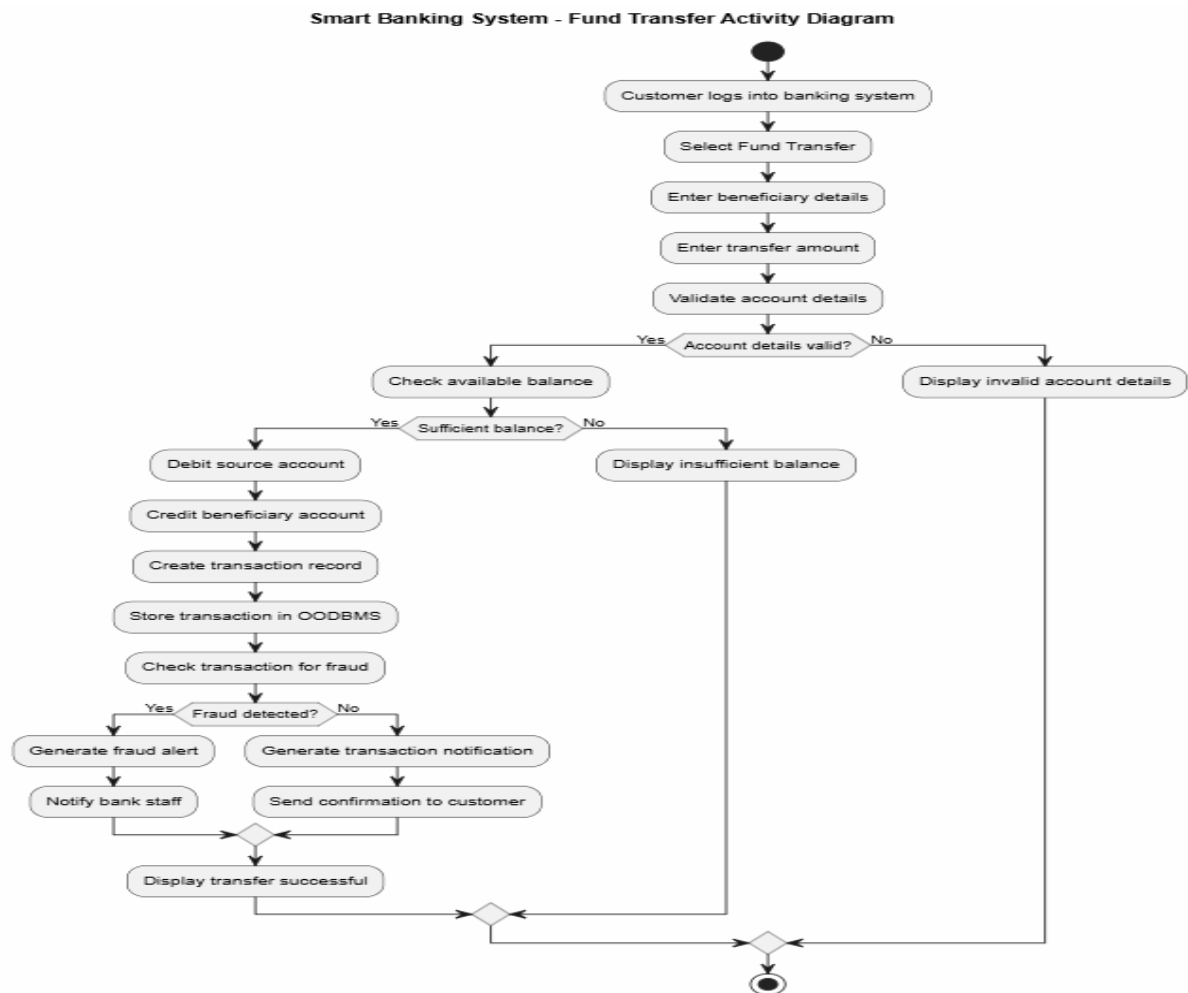
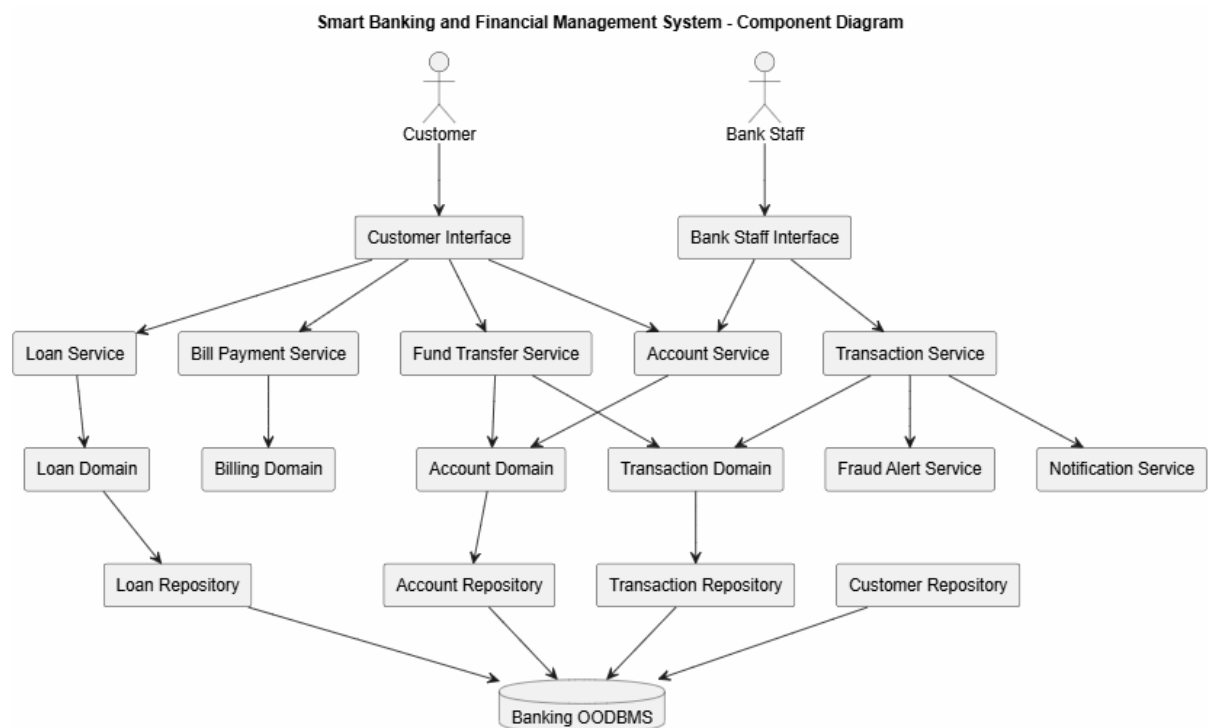


Diagram 7: Component Diagram



Scalability, Security, Flexibility and Maintainability

Requirement	Design Approach
Scalability	New services can be added independently
Security	Layered access and fraud monitoring
Flexibility	Factory and Strategy patterns
Maintainability	Separation of responsibilities
Extensibility	Supports future investment and digital-wallet services

Conclusion

The proposed Smart Banking System uses OOP principles, layered architecture, Factory, Strategy, Observer and Repository patterns, and OODBMS persistence to create a scalable and maintainable banking solution.

The architecture separates presentation, business logic, domain objects, and persistence, while allowing future services such as investment management and digital wallets to be added without major redesign.

2. Smart Retail and Inventory Management System

A retail organization develops a **Smart Retail and Inventory Management System** to manage products, suppliers, customers, purchase orders, sales transactions, and warehouse stock. The system follows a layered object-oriented architecture with presentation, business, domain, and data access layers connected to an OODBMS.

The system applies the **MVC Pattern** for managing user interfaces, the **Factory Pattern** for creating different product categories, and the **Observer Pattern** to notify managers when inventory reaches a minimum threshold. The **DAO Pattern** handles communication with persistent inventory objects. The architecture allows future integration of automated stock prediction and robotic warehouse management.

Question:

Evaluate how object-oriented design principles, layered architecture, and design patterns improve the flexibility, scalability, and maintainability of the retail system. Explain how the different layers interact with the OODBMS for efficient inventory persistence and retrieval.

Introduction

The Smart Retail and Inventory Management System manages products, suppliers, customers, purchase orders, sales transactions, and warehouse stock. It follows a layered object-oriented architecture consisting of Presentation, Business, Domain, and Data Access layers, connected to an OODBMS.

The system uses:

- **MVC Pattern** for user interfaces
- **Factory Pattern** for creating product categories

- **Observer Pattern** for minimum-stock notifications
- **DAO Pattern** for persistent inventory objects

Future extensions include automated stock prediction and robotic warehouse management.

Object-Oriented Principles

Encapsulation

Product information such as product ID, price, and quantity is kept private and accessed through methods.

Abstraction

Interfaces such as ProductFactory, InventoryObserver, and InventoryDAO hide implementation details.

Inheritance

Different product categories can inherit common properties and operations from a base Product class.

Polymorphism

Different product categories can implement common operations differently while being accessed through the same base type.

Reusability

Common product, inventory, and transaction operations can be reused across different retail modules.

Diagram 1: Layered Architecture

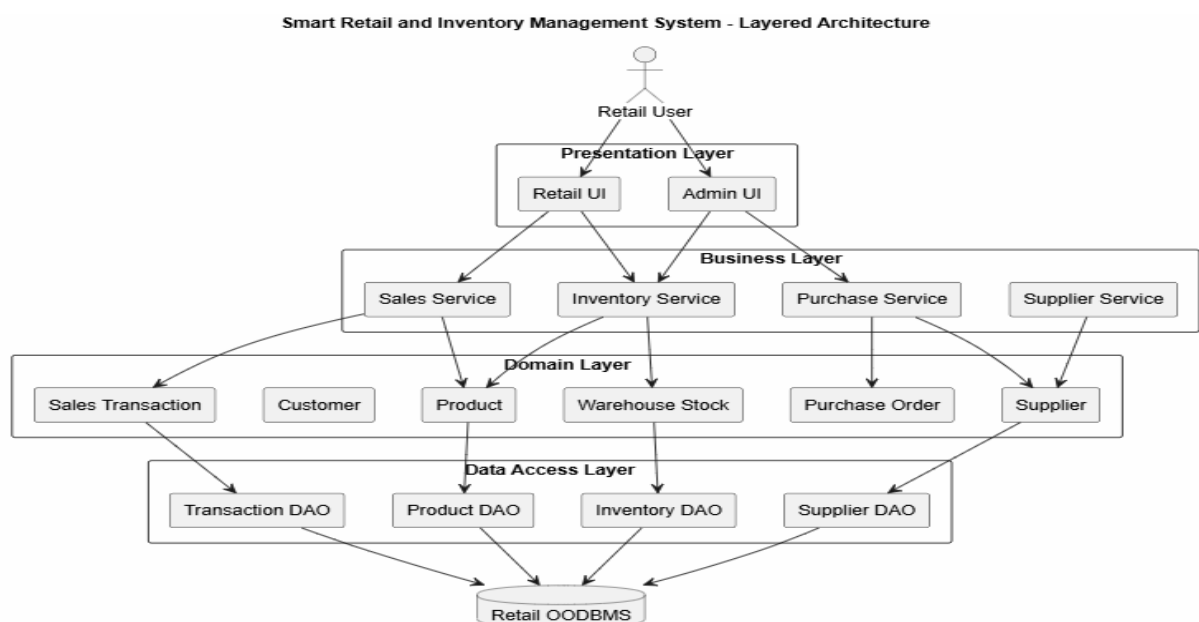


Diagram 2: Class Diagram with Design Patterns

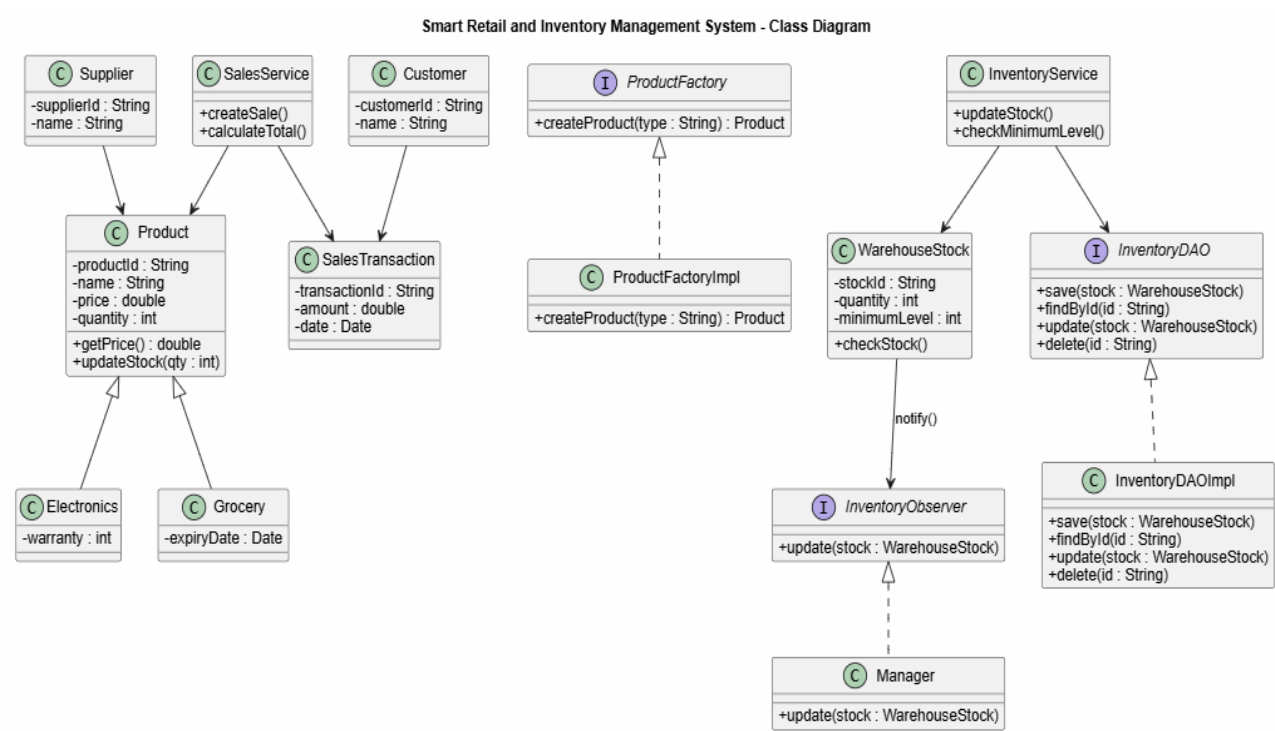


Diagram 3: Inventory Stock Alert Sequence Diagram

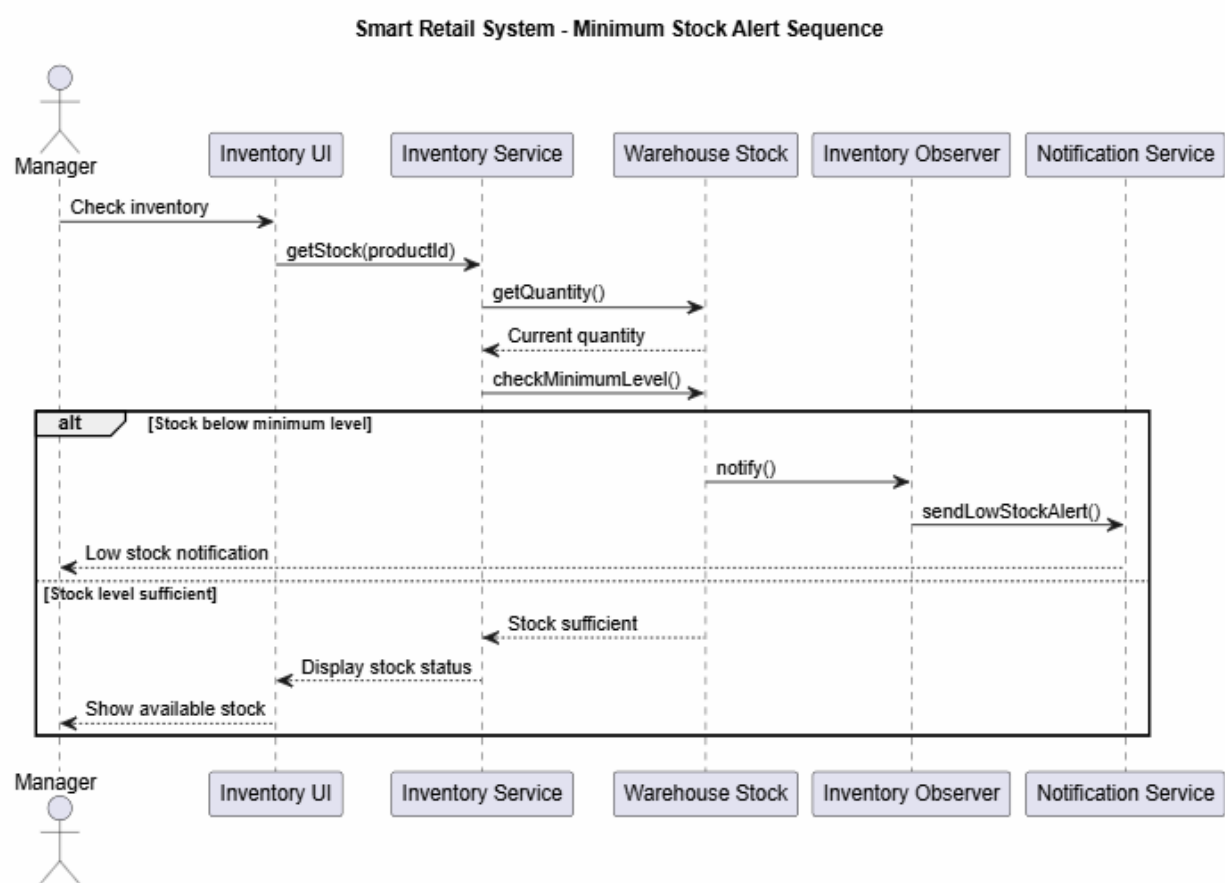


Diagram 4: OODBMS Inventory Persistence Sequence Diagram

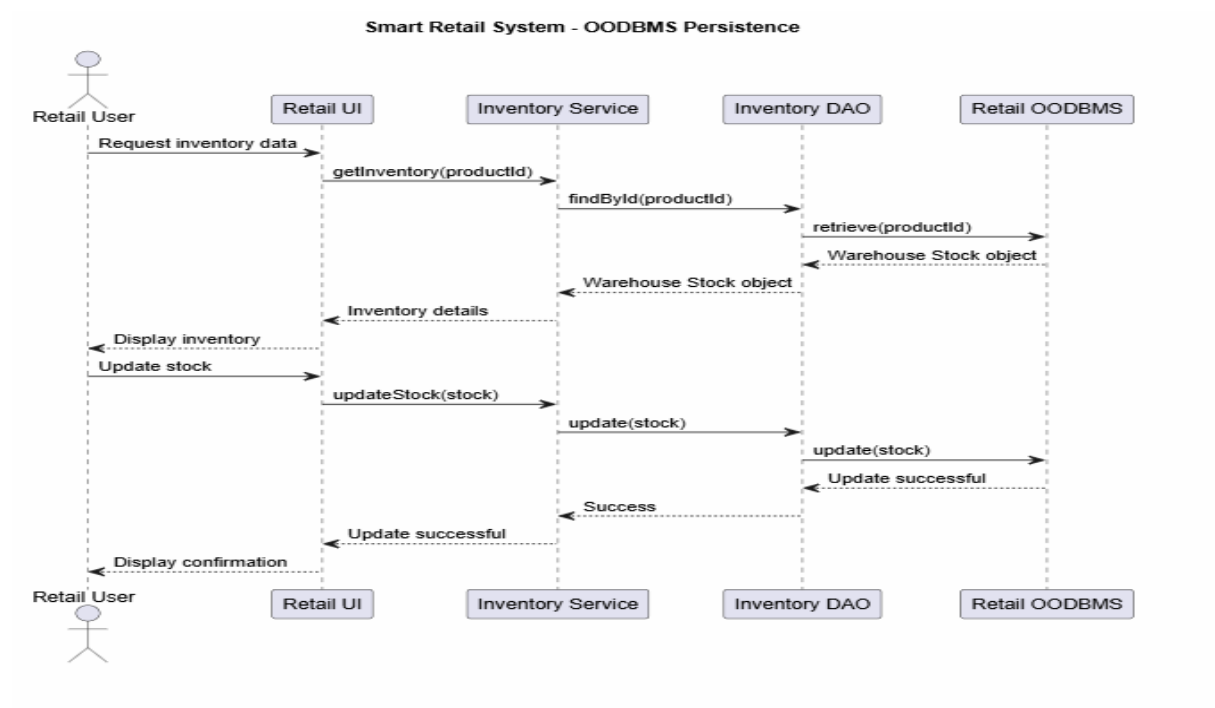
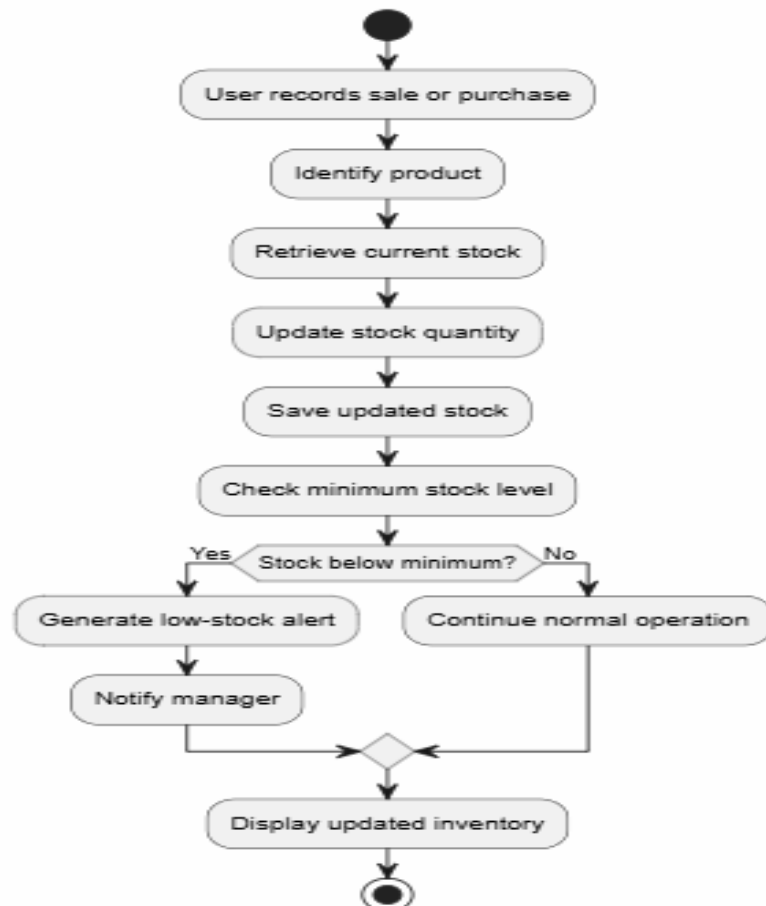


Diagram 5: Inventory Management Activity Diagram

Smart Retail System - Inventory Management Activity



Conclusion

The Smart Retail and Inventory Management System uses object-oriented principles and layered architecture to separate user interaction, business operations, domain objects, and persistence.

The MVC Pattern improves UI management, the Factory Pattern simplifies product creation, the Observer Pattern provides minimum-stock notifications, and the DAO Pattern separates inventory persistence from business logic.

The architecture improves flexibility, scalability, and maintainability and provides a foundation for future automated stock prediction and robotic warehouse management.

3. Smart Hotel and Hospitality Management System

A hotel chain develops a **Smart Hotel Management System** to manage guest registration, room reservations, check-in/check-out, payments, room services, and customer feedback. The application uses a layered object-oriented architecture consisting of presentation, service, domain, and data access layers connected to an OODBMS.

The system implements the **Strategy Pattern** for different pricing and discount policies, the **Factory Pattern** for creating room and service objects, and the **Observer Pattern** for reservation and service notifications. A **Repository Pattern** is used to simplify communication between business components and persistent hotel objects. Future enhancements include smart-room automation and personalized guest services.

Question:

Analyze how object-oriented principles and design patterns support flexibility, reusability, and maintainability in the hotel management system. Explain how the layered architecture coordinates reservation processing and communicates with the OODBMS for persistent data management.

Introduction

The Smart Hotel Management System manages guest registration, room reservations, check-in/check-out, payments, room services, and customer feedback. It uses a layered object-oriented architecture consisting of Presentation, Service, Domain, and Data Access layers, connected to an OODBMS.

The system uses:

- **Strategy Pattern** for pricing and discount policies.
- **Factory Pattern** for creating room and service objects.
- **Observer Pattern** for reservation and service notifications.
- **Repository Pattern** for communication with persistent hotel objects.

Future enhancements include smart-room automation and personalized guest services.

Object-Oriented Principles

Encapsulation

Guest, room, reservation, and payment data are kept private and accessed through methods.

Abstraction

Interfaces such as PricingStrategy, RoomFactory, and ReservationObserver hide implementation details.

Inheritance

Different room types can inherit common properties from a base Room class.

Polymorphism

Different pricing or discount strategies can be used through the same PricingStrategy interface.

Reusability

Common hotel operations can be reused across different room types, services, and reservation processes.

Diagram 1: Layered Architecture

Smart Hotel and Hospitality Management System - Layered Architecture

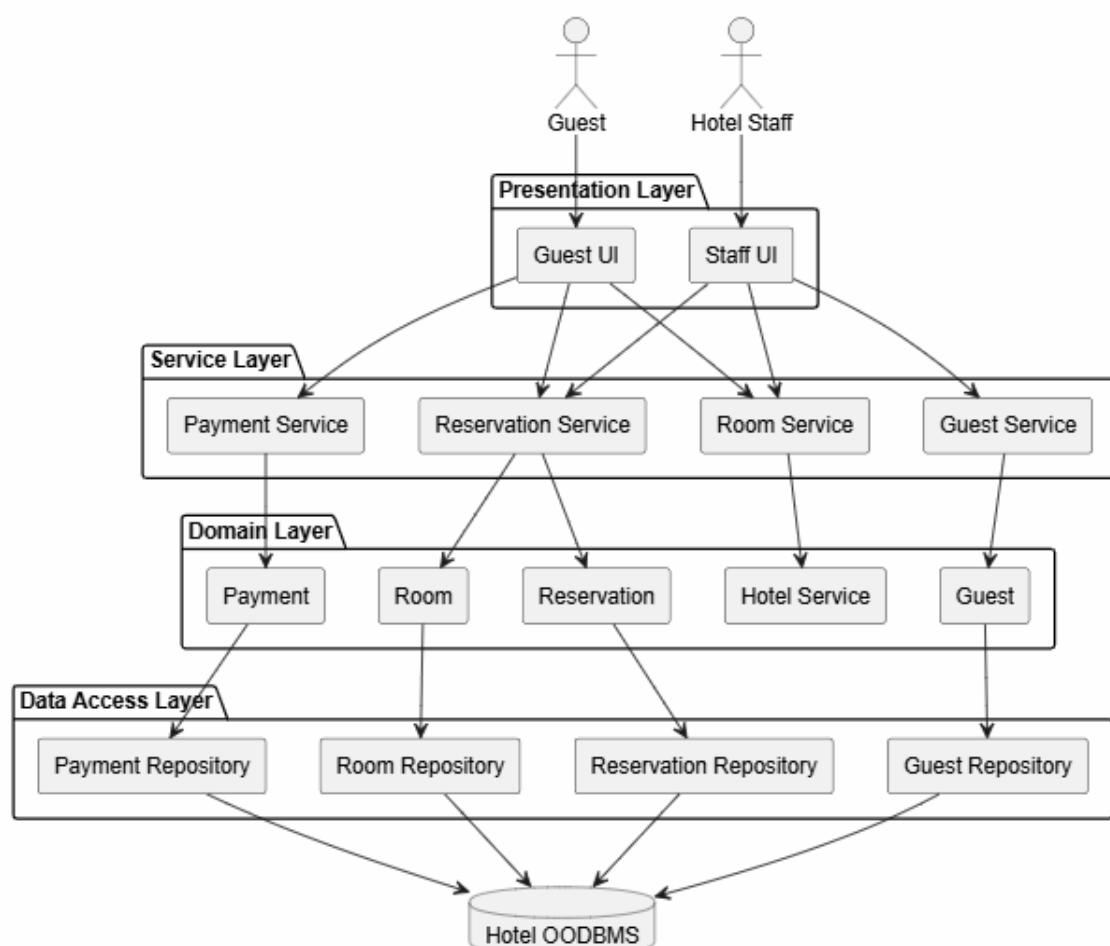


Diagram 2: Class Diagram with Design Patterns

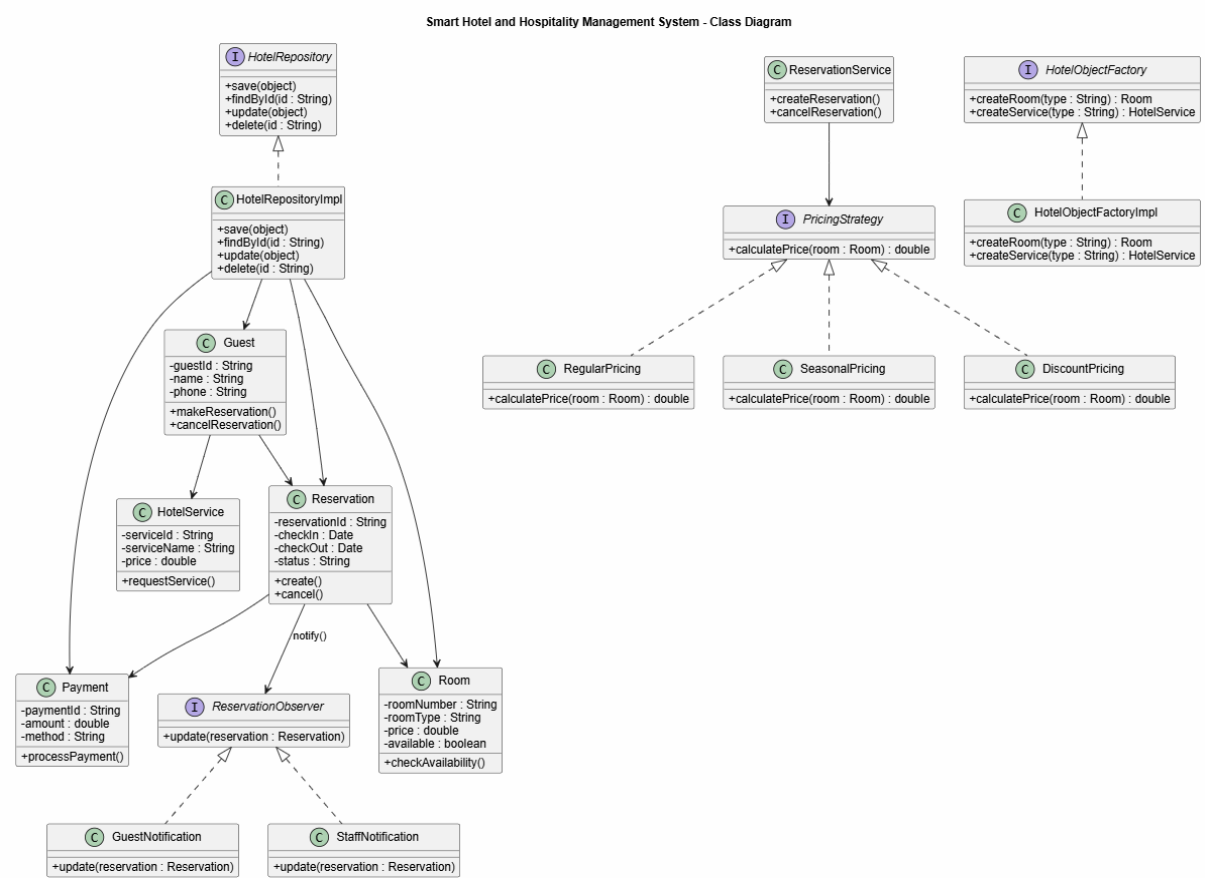


Diagram 3: Room Reservation Sequence Diagram

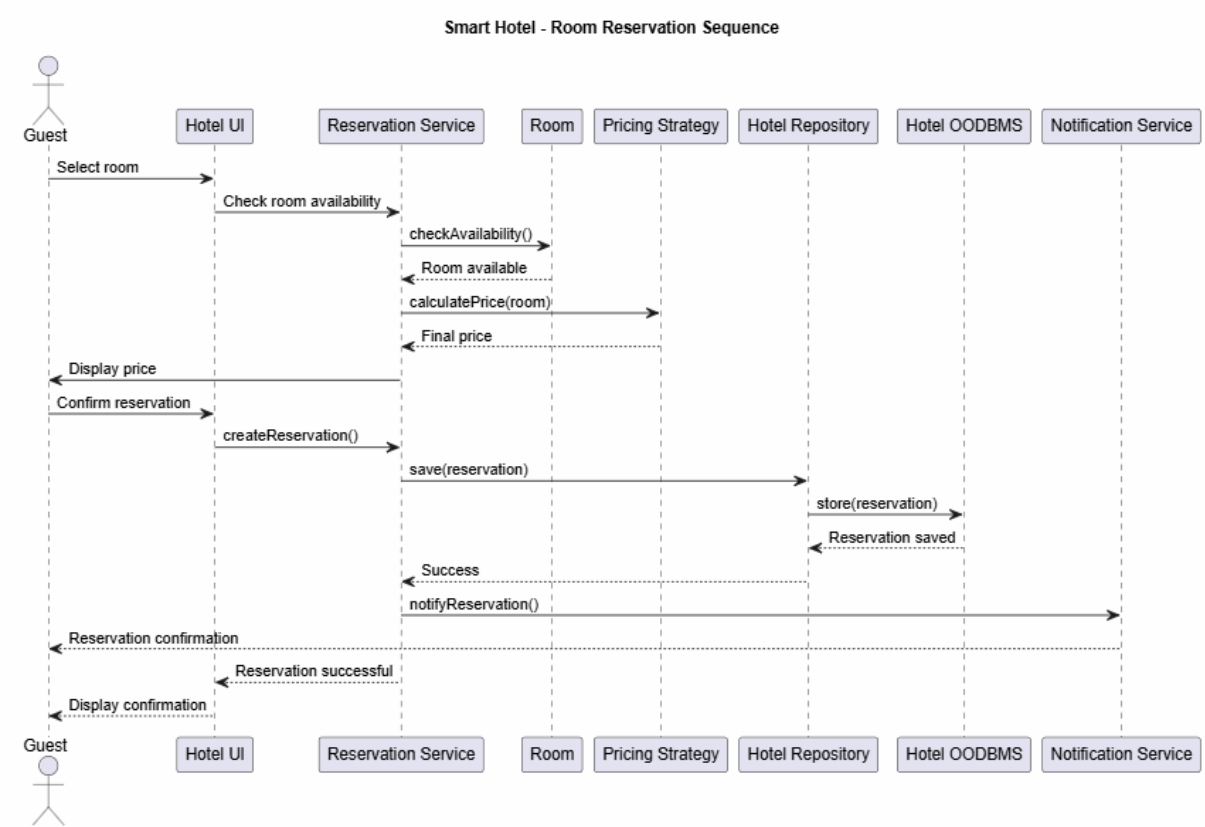


Diagram 4: OODBMS Persistence Sequence Diagram

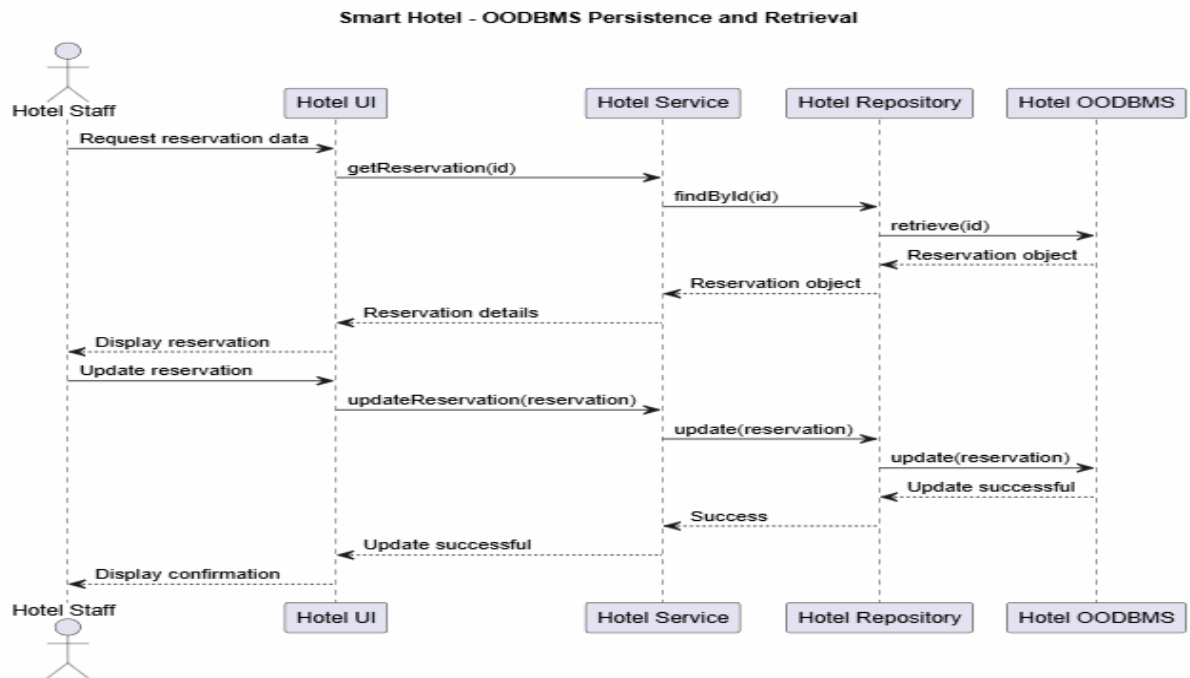
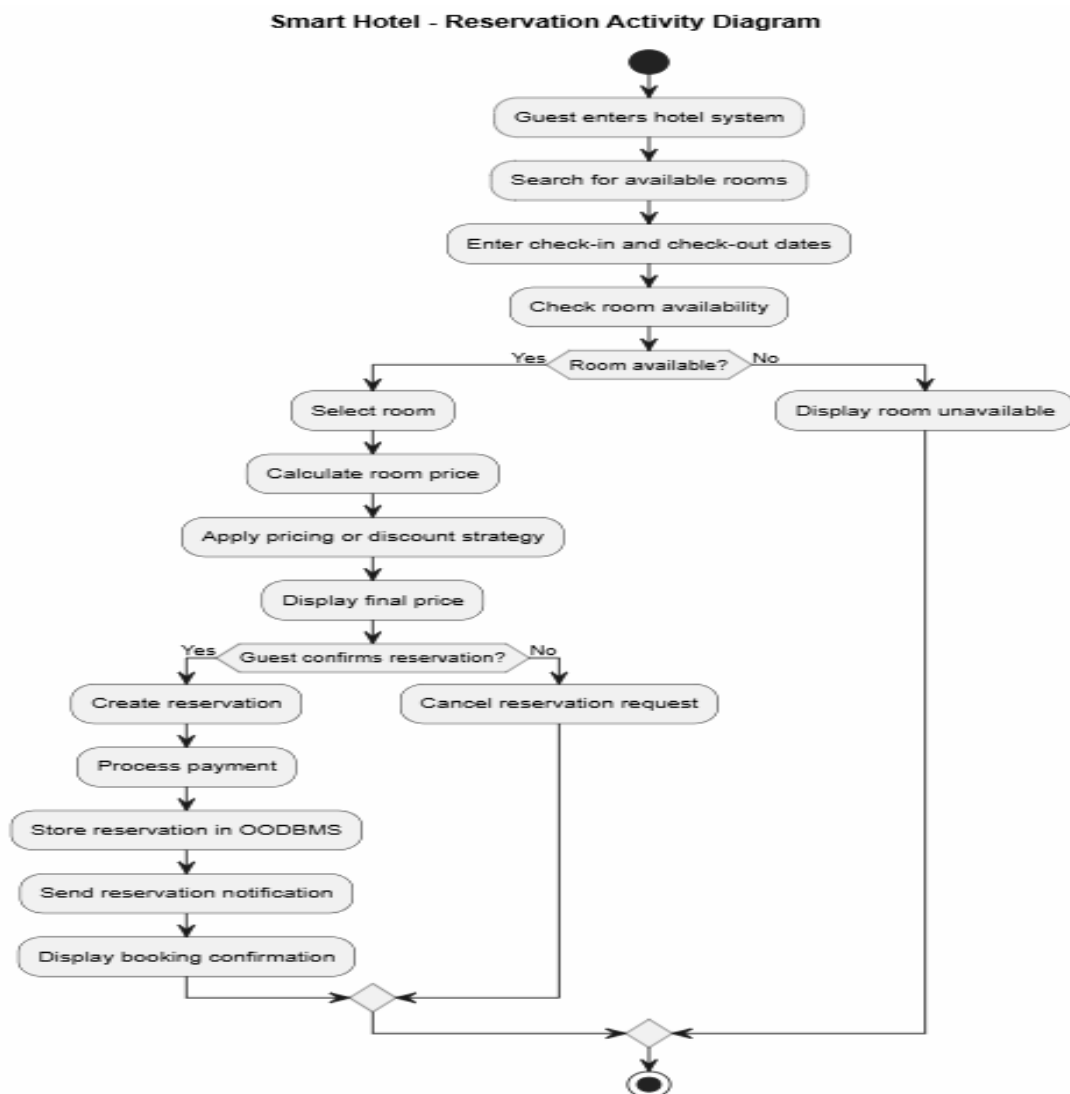


Diagram 5: Hotel Reservation Activity Diagram



Flexibility, Reusability and Maintainability

Requirement	Design Solution
Flexibility	Strategy Pattern for pricing
Reusability	Common Room and Service classes
Maintainability	Layered architecture
Object creation	Factory Pattern
Notifications	Observer Pattern
Persistence	Repository + OODBMS
Future expansion	Smart-room and personalized services

Conclusion

The Smart Hotel Management System uses object-oriented principles, layered architecture, and design patterns to manage reservations, rooms, payments, services, and guest information.

The Strategy Pattern provides flexible pricing and discount policies, the Factory Pattern simplifies creation of room and service objects, the Observer Pattern handles reservation and service notifications, and the Repository Pattern manages persistent hotel objects.

The architecture improves flexibility, reusability, and maintainability while allowing future enhancements such as smart-room automation and personalized guest services.

4. Smart Logistics and Delivery Management System

A logistics company develops a **Smart Logistics and Delivery Management System** to manage shipment registration, warehouse operations, vehicle allocation, delivery tracking, and customer notifications. The system adopts a layered object-oriented architecture with presentation, business, domain, and data access layers connected to an OODBMS.

The system uses the **Strategy Pattern** for selecting optimal delivery routes, the **Factory Pattern** for creating different shipment types, and the **Observer Pattern** for real-time delivery status notifications. A **DAO Pattern** manages persistent shipment and vehicle information. The architecture is designed to support future integration with autonomous delivery vehicles and AI-based route optimization.

Question:

Examine how object-oriented design principles, layered architecture, and design patterns improve the adaptability, scalability, and maintainability of the logistics system. Explain how the layers communicate with the OODBMS to manage persistent shipment, vehicle, and delivery information.

Introduction

The Smart Logistics and Delivery Management System manages shipment registration, warehouse operations, vehicle allocation, delivery tracking, and customer notifications. It uses a layered object-oriented architecture consisting of Presentation, Business, Domain, and Data Access layers, connected to an OODBMS.

The system uses:

- **Strategy Pattern** for selecting optimal delivery routes.
- **Factory Pattern** for creating different shipment types.
- **Observer Pattern** for real-time delivery-status notifications.
- **DAO Pattern** for persistent shipment and vehicle information.

Future extensions include autonomous delivery vehicles and AI-based route optimization.

Object-Oriented Principles

Encapsulation

Shipment, vehicle, and delivery information is kept inside objects and accessed through defined methods.

Abstraction

Interfaces such as RouteStrategy, ShipmentFactory, DeliveryObserver, and ShipmentDAO hide implementation details.

Inheritance

Different shipment types and vehicle types can inherit common properties from base classes.

Polymorphism

Different route strategies can be selected through a common RouteStrategy interface.

These principles reduce coupling and make the logistics system easier to extend.

Diagram 1: Layered Architecture

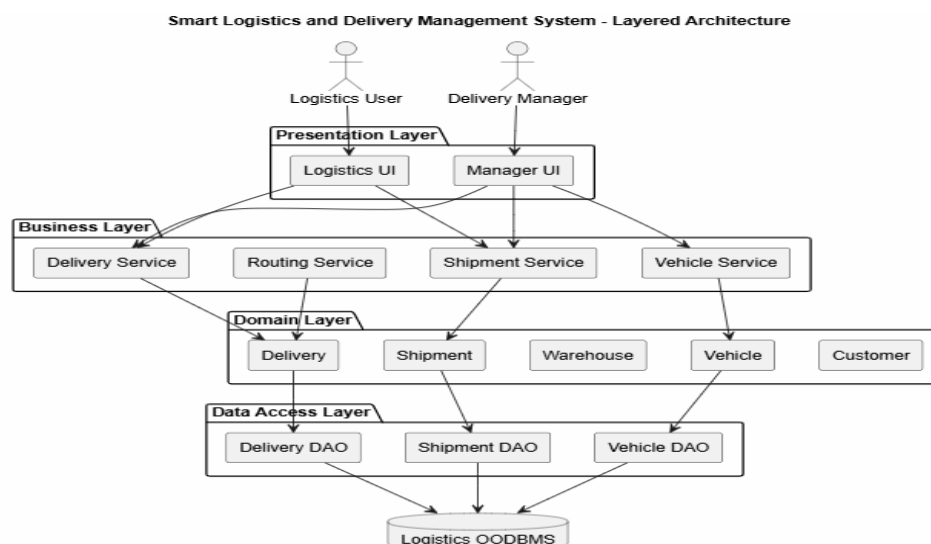


Diagram 2: Class Diagram with Design Patterns

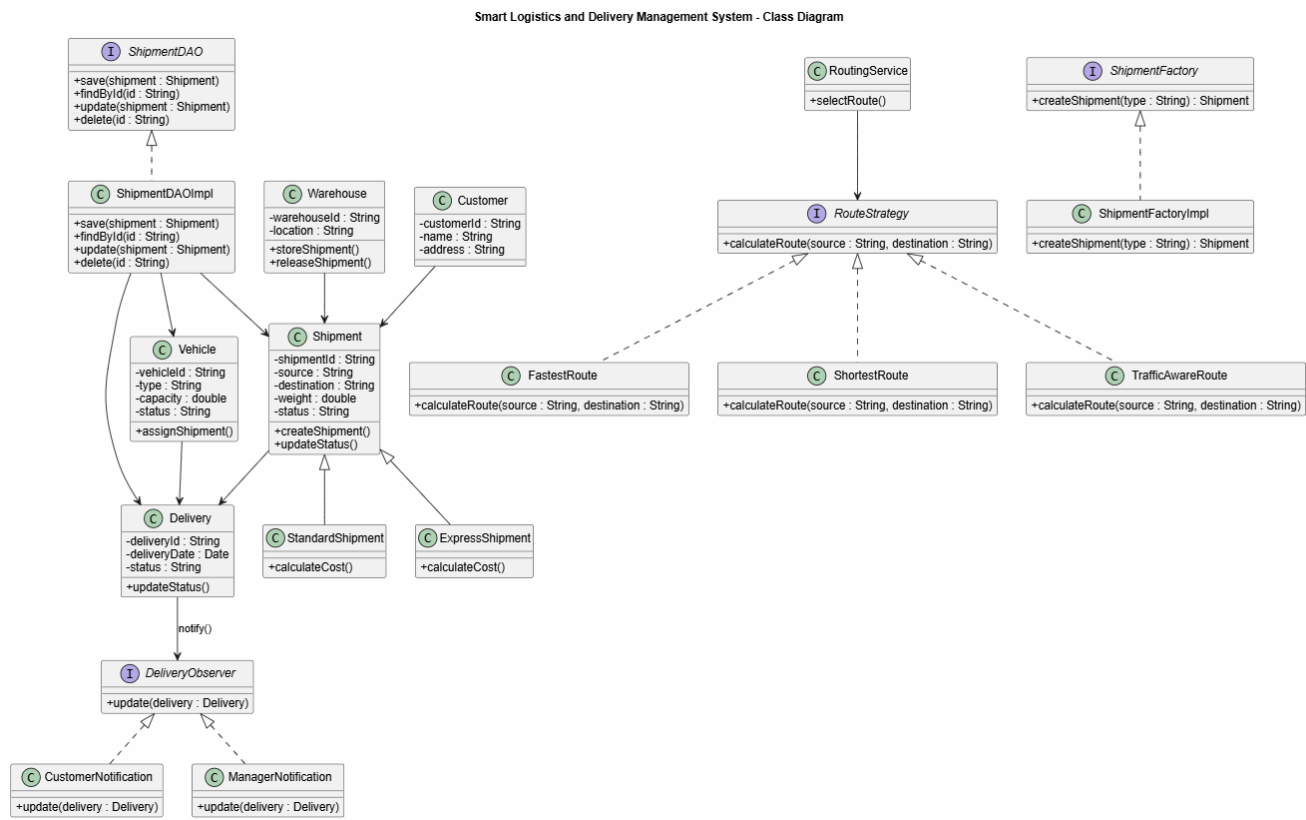


Diagram 3: Delivery Tracking Sequence Diagram

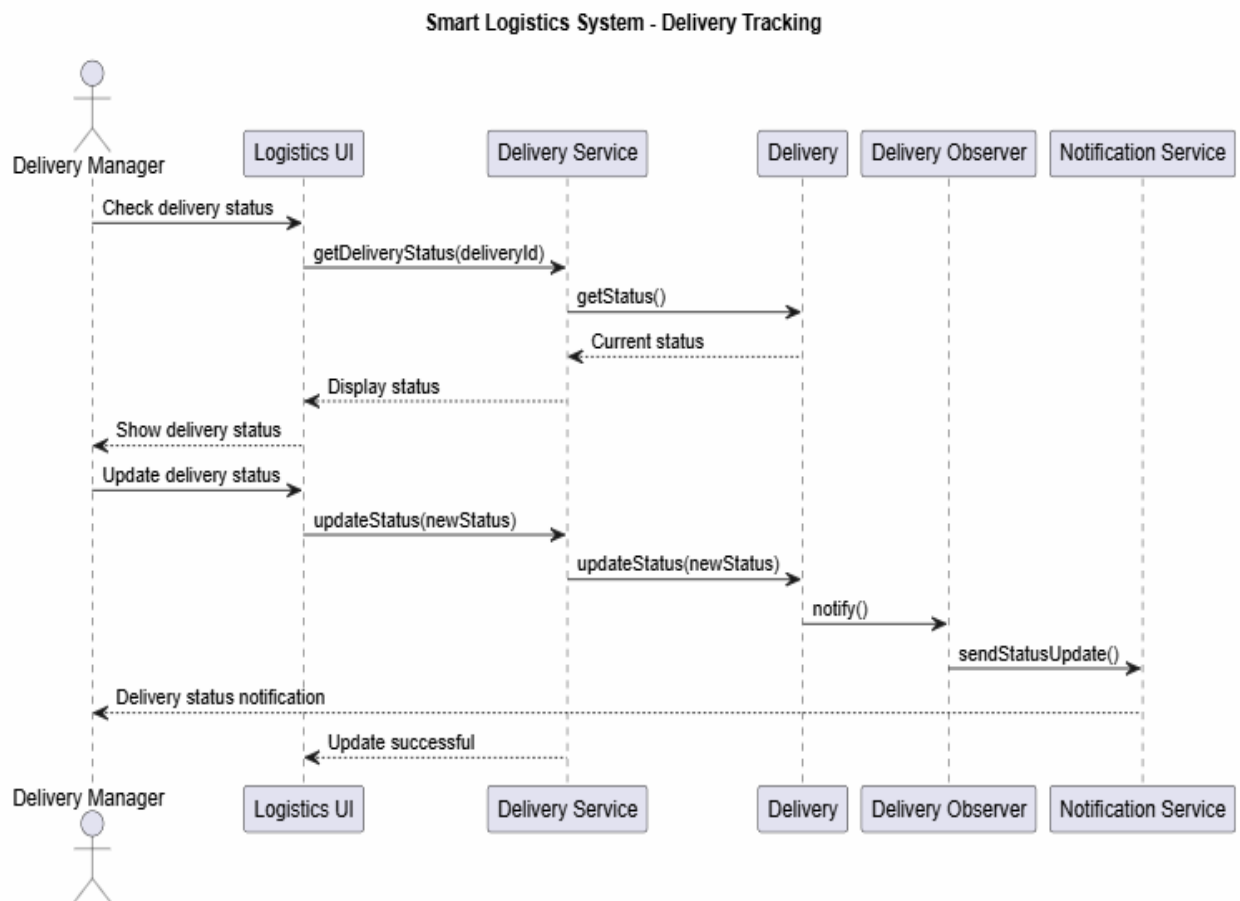


Diagram 4: OODBMS Persistence Sequence Diagram

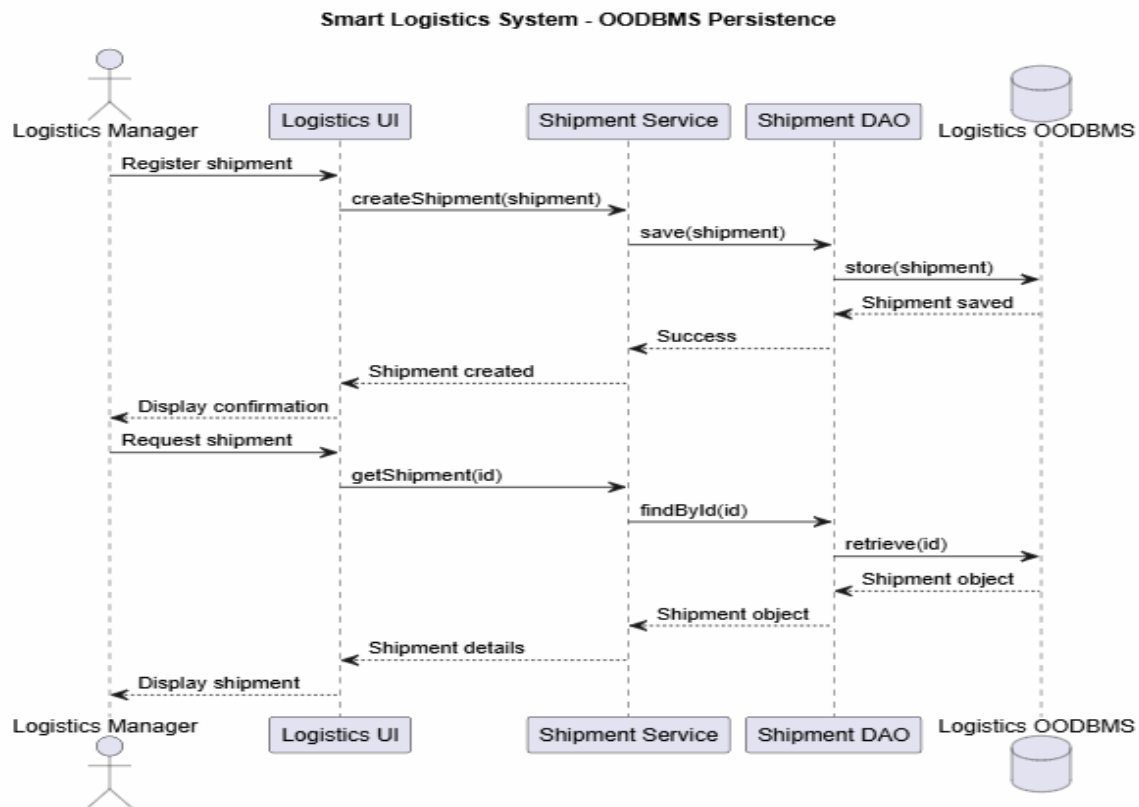


Diagram 5: Shipment Delivery Activity Diagram



Conclusion

The Smart Logistics and Delivery Management System uses object-oriented principles, layered architecture, and design patterns to efficiently manage shipments, warehouses, vehicles, and deliveries.

The Strategy Pattern provides flexible route selection, the Factory Pattern simplifies shipment creation, the Observer Pattern enables real-time delivery notifications, and the DAO Pattern manages persistent shipment and vehicle information.

The architecture improves adaptability, scalability, and maintainability while providing a foundation for future autonomous delivery vehicles and AI-based route optimization.

5. Smart Energy Management System

An energy management company develops a **Smart Energy Management System** to monitor electricity consumption, manage smart meters, analyze energy usage, and generate alerts for abnormal consumption. The system follows a layered object-oriented architecture where the presentation layer provides dashboards, the business layer performs energy analysis, the domain layer manages energy-related rules, and the data access layer communicates with an OODBMS.

The system uses the **Observer Pattern** for consumption alerts, the **Strategy Pattern** for different energy optimization algorithms, and the **Factory Pattern** for creating different smart-meter objects. A **Repository Pattern** is used to manage persistent energy data. The architecture is designed to accommodate future renewable-energy monitoring and AI-based consumption prediction.

Question:

Evaluate how object-oriented principles, layered architecture, and design patterns contribute to the scalability, extensibility, and maintainability of the energy management system. Explain the interaction between the system layers and the OODBMS for storing, retrieving, and managing persistent energy-monitoring objects.

Introduction

The Smart Energy Management System monitors electricity consumption, manages smart meters, analyzes energy usage, and generates alerts for abnormal consumption. It follows a layered object-oriented architecture consisting of Presentation, Business, Domain, and Data Access layers, connected to an OODBMS.

The system uses:

- **Observer Pattern** for abnormal-consumption alerts.
- **Strategy Pattern** for different energy optimization algorithms.
- **Factory Pattern** for creating different smart-meter objects.

- **Repository Pattern** for managing persistent energy data.

Future extensions include renewable-energy monitoring and AI-based consumption prediction.

Object-Oriented Principles

Encapsulation

Energy consumption, meter readings, and energy rules are kept inside their respective objects and accessed through methods.

Abstraction

Interfaces such as OptimizationStrategy, MeterFactory, and EnergyRepository hide implementation details.

Inheritance

Different smart-meter types can inherit common properties from a base SmartMeter class.

Polymorphism

Different optimization algorithms can be used through the same OptimizationStrategy interface.

Reusability

Common meter and energy-management operations can be reused across different energy services.

Diagram 1: Layered Architecture

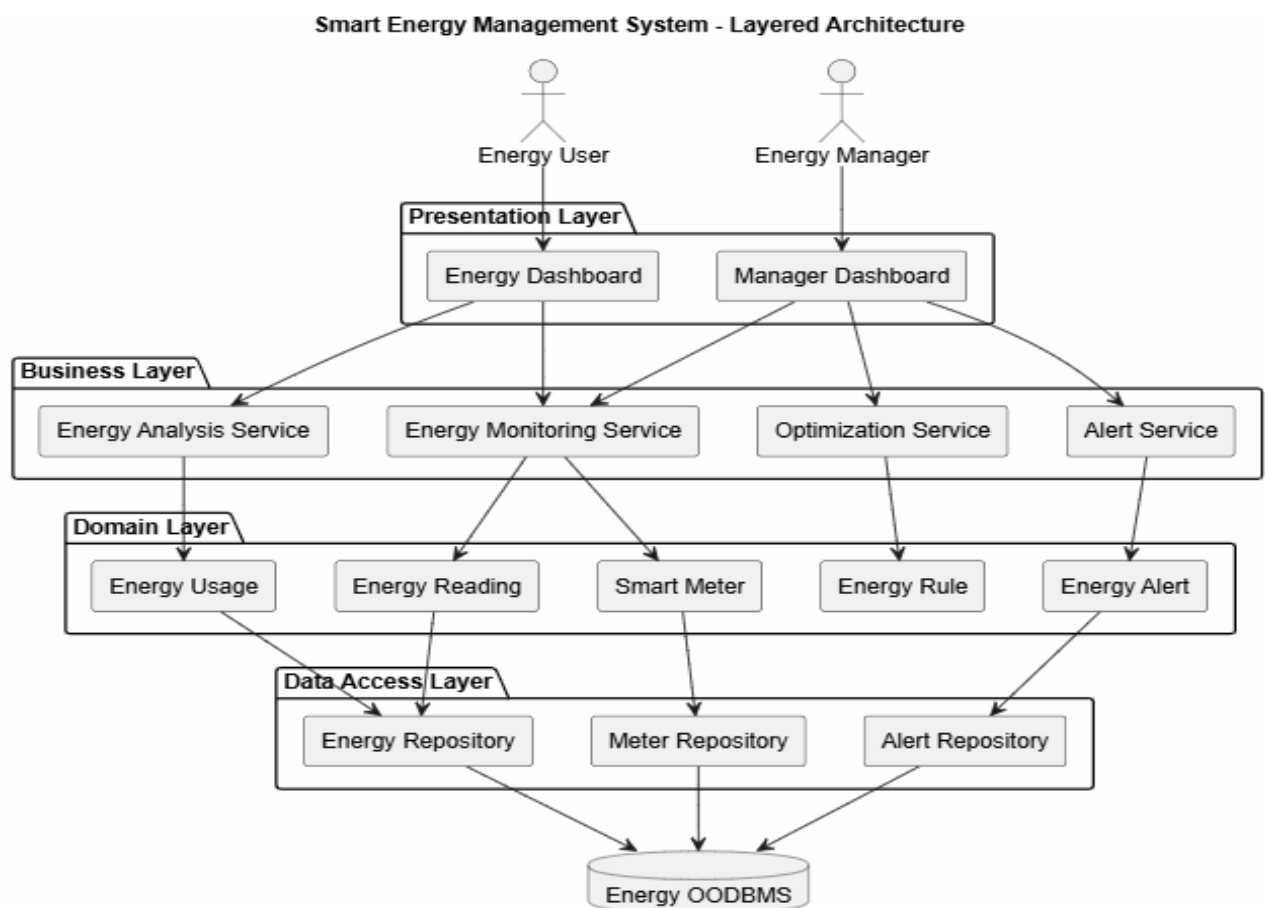


Diagram 2: Class Diagram

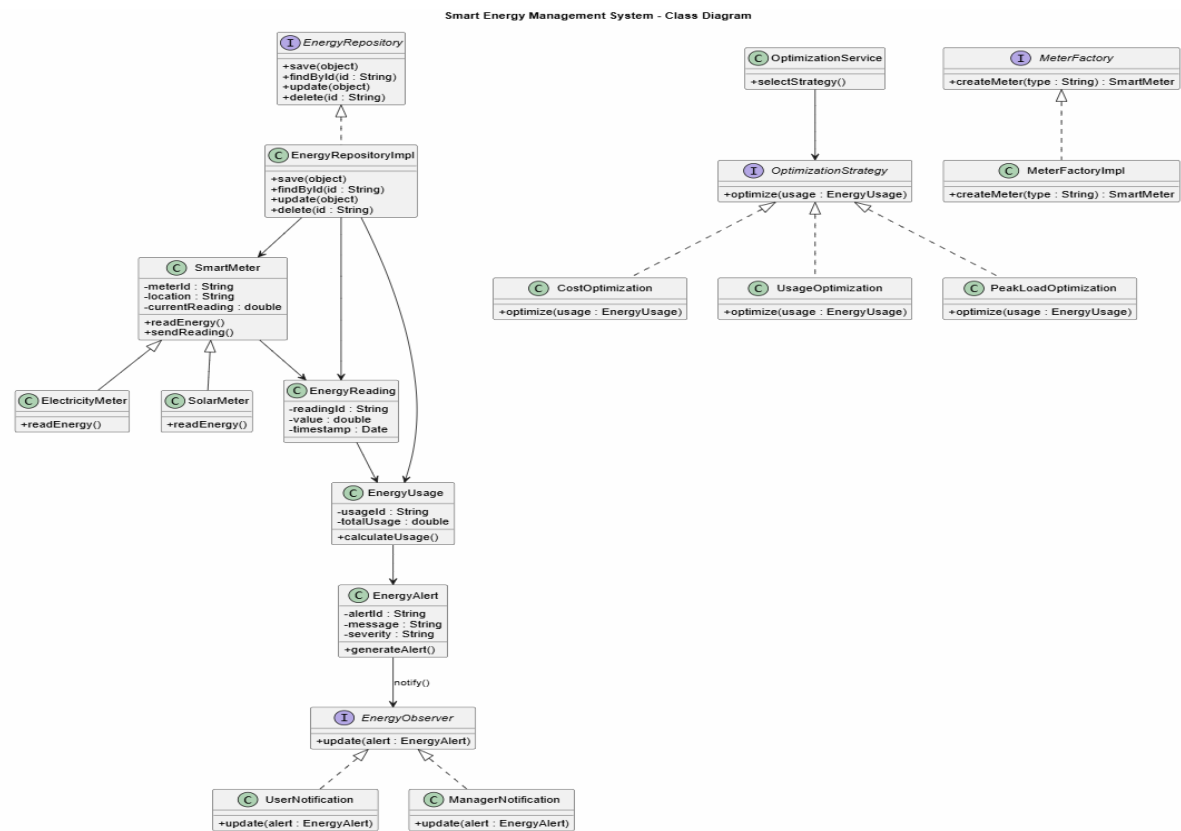


Diagram 3: Abnormal Consumption Alert

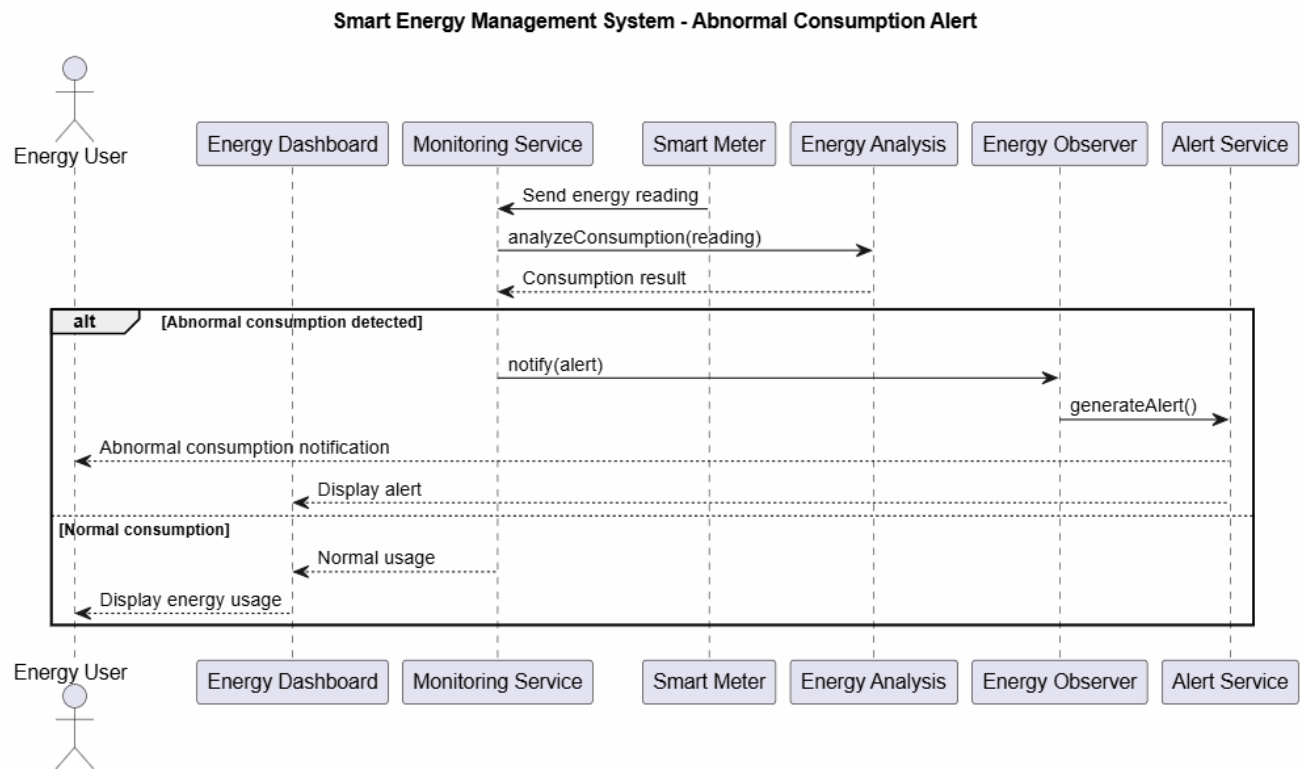


Diagram 4: OODBMS Energy Data Persistence

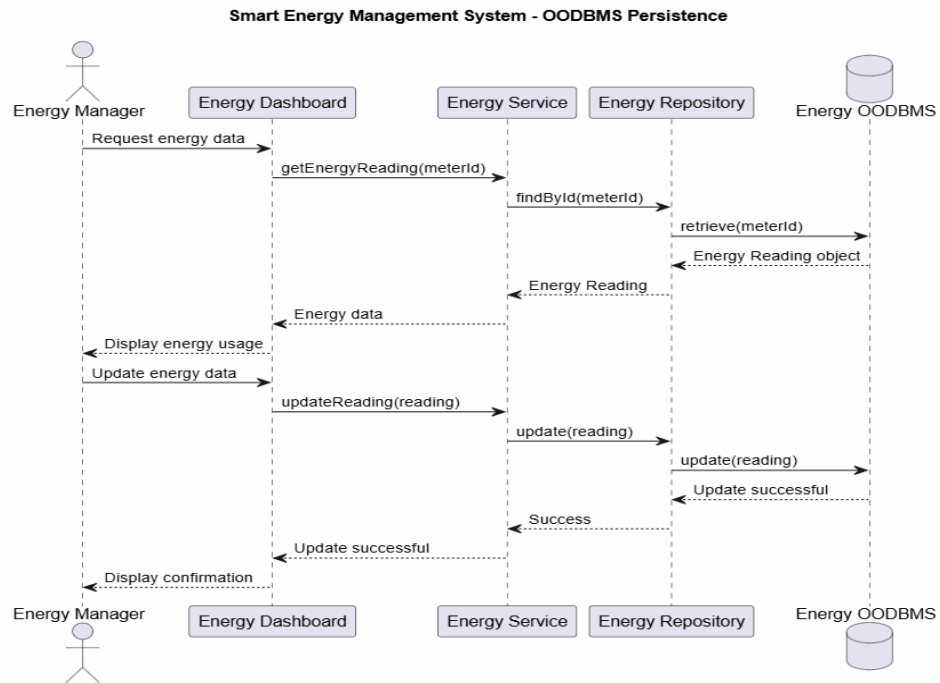
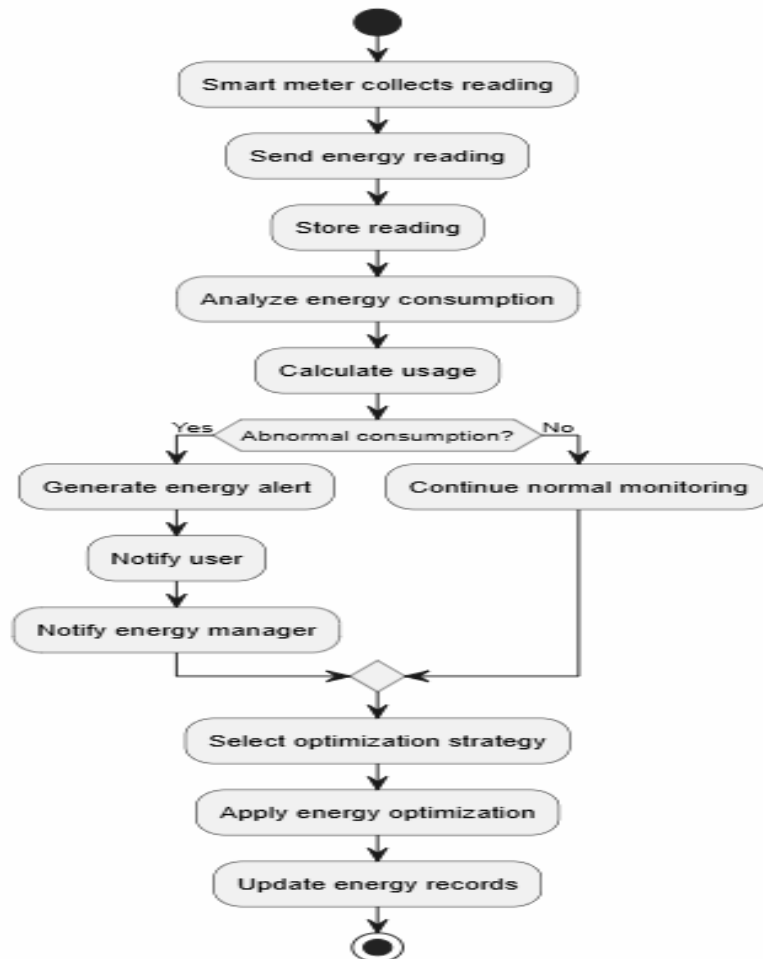


Diagram 5: Energy Monitoring Workflow

Smart Energy Management System - Energy Monitoring Workflow



Conclusion

The Smart Energy Management System uses object-oriented principles, layered architecture, and design patterns to monitor and analyze energy consumption.

The Observer Pattern provides abnormal-consumption alerts, the Strategy Pattern supports different energy optimization algorithms, the Factory Pattern simplifies smart-meter creation, and the Repository Pattern manages persistent energy data.

The architecture improves scalability, extensibility, and maintainability while supporting future renewable-energy monitoring and AI-based consumption prediction.

RUBRICS FOR CALCULATION

Industry Problem Solving Task

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding ; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation